

Credit Batch Reconciler PRD

PRD: Credit Batch Reconciler (Desktop App)

Owner: Patel **Status:** Planning — replaces browser-only exporter roadmap **Last updated:** July 2026 **Supersedes:** PRD_batch_report_exporter.md (Phase 1 prototype remains the extraction reference implementation)

1. Problem

Gas station operators receive **Chevron credit batch settlement PDFs** and **EFT prenotification invoice PDFs** from their processor. Today they:

- Manually re-key batch totals into Excel workbooks (slow, error-prone)
- Maintain one spreadsheet per store with batches, invoice amounts, and reconcile notes mixed together
- Cannot reliably detect when an **invoice is missing credit batches**, because reconciliation only works against whatever PDF or Excel slice was loaded in that session — not the full history for the store

With **multiple stores**, the problem multiplies: each store has its own batches and invoices, and batch numbers can collide across stores.

There is no purpose-built local tool that **accumulates batch history**, **stores invoice data**, **flags missing batches**, and **exports a clean Excel report** for review.

2. Goals

- **Persist all credit batches per store** in a local SQLite database (one `.db` file per store).
 - **Ingest Chevron settlement PDFs** and append extracted batches with deduplication.
 - **Ingest EFT invoice PDFs** and store invoice headers and line items.
 - **Reconcile bidirectionally**: flag batches missing from an invoice, invoice lines with no matching batch, and amount mismatches.
 - **Compute file-level Credit discrepancy** (invoice total vs matched batch nets) when reconciliation coverage is complete.
 - **Export to Excel** for human review — export is a **report**, not the system of record.
 - Ship as a **distributable desktop app** (Windows `.exe` for client PCs; develop and test on Mac).
 - Preserve the **layered architecture** from the prototype so new PDF templates are additions, not rewrites.
-

3. Non-goals

- Cloud hosting, accounts, or multi-user sync server
 - Accounting software integration (QuickBooks, etc.)
 - OCR / scanned-PDF support
 - Automatic detection of unknown PDF formats or self-serve template builder
 - Excel or OneDrive as the **live ledger** (read-modify-write on shared `.xlsx` files)
 - Cross-store reconciliation in a single run (each store is isolated by its own `.db`)
 - Mobile or web deployment in v1
-

4. Target users

- Gas station owners and operators managing **one or more stores**
- Comfortable double-clicking an app and uploading PDFs; **not** comfortable with SQL, regex, or manual spreadsheet maintenance

- May want to open exported Excel on OneDrive for review; the `.db` files stay on the machine that runs the app (or a single canonical data folder)

5. Guiding principles

5.1 PDF variation is the core risk

Processor PDF layouts change. Isolate format-specific logic in **templates**; never leak template concerns into storage or export layout.

Layer	Responsibility	Changes when...
Ingestion	Read PDF bytes → text lines	Never
Extraction	Per-processor template parsers	New processor or format
Normalization	Fixed internal record schema	Never (contract)
Storage	SQLite read/write per store	Schema version migrations only
Reconciliation	Join batches ↔ invoice lines; flags	Match rules refined
Output	Excel export layout	Output requirements change

Folder mapping:

Layer	Backend folder
Ingestion + Extraction + Normalization	backend/parsing/
Storage + Reconciliation	backend/reconciling/
Excel output	backend/exporting/
UI (screens, IPC calls, previews)	frontend/

Rule: Adding a new PDF template must not require changing the Excel export column spec or the SQLite schema (only extraction + mapping). New templates go in `backend/parsing/templates/` only.

5.2 SQLite is the ledger; Excel is the view

- **Source of truth:** `{StoreName}.db` on disk
- **Excel:** Generated on demand after ingest or reconcile; safe to save to OneDrive as read-only snapshots
- Do not parse exported Excel back into the ledger except during a **one-time legacy import** (Phase 6)

5.3 One database per store

- File naming: `Sunset.db`, `Mako.db`, etc.
- Store identity from Chevron `site_id` at ingest; user selects store in the UI before operations
- Reconciliation never crosses databases

6. Technology stack

Component	Choice	Notes
Desktop shell	Electron	Reuse existing HTML/JS UI; Node.js main process for filesystem + SQLite
SQLite	better-sqlite3	Synchronous, reliable local file access
PDF text extraction	pdf.js	Already used in prototype
Excel export	SheetJS (xlsx)	Already used in prototype; write-only
Packaging	electron-builder	Mac <code>.app</code> for dev; Windows <code>.exe</code> via GitHub Actions or Windows build machine

Tests	Node (existing)	Parser + reconcile unit tests; add storage integration tests
-------	------------------------	--

6.1 Development vs distribution

Activity	Where
Daily development	Mac (Apple Silicon)
Local testing	Mac .app
Windows installer	GitHub Actions Windows runner or Windows PC/VM (required for native better-sqlite3 Windows binaries)

6.2 Repository layout

The repo has **three top-level folders**. Business logic lives in `backend/` ; the renderer never imports parsing or SQLite directly — it calls the backend through Electron IPC.

```
credit-batch-software/
├─ frontend/           # Everything UI-related (renderer process)
├─ backend/           # Application logic (main process + modules)
│  ├─ parsing/        # PDF → normalized records
│  │  ├─ reconciling/  # SQLite ledger + match engine
│  │  └─ exporting/    # SQLite / records → .xlsx
├─ documentation/     # PRDs, architecture notes, user guides
├─ test/              # Mirrors backend folder layout
├─ package.json        # Root scripts: start, test, build
└─ .github/workflows/  # Windows installer CI (Phase 6)
```

Dependency rule: `frontend/` → IPC only → `backend/` . `backend/parsing/` → `backend/reconciling/` is allowed (ingest then save). `backend/reconciling/` → `backend/exporting/` is allowed (export after read). `backend/parsing/` must **not** import from `exporting/` or `reconciling/` .

7. Data model (SQLite — per store `.db`)

7.1 Schema version

Each database includes a `schema_version` table (single row). Migrations run on open when version is behind.

7.2 Table: `batches`

Settlement data from Chevron PDFs.

Column	Type	Notes
<code>id</code>	INTEGER PK	Auto-increment
<code>site_id</code>	TEXT	From Chevron PDF
<code>batch_date</code>	TEXT (ISO date)	<code>YYYY-MM-DD</code>
<code>batch_number</code>	TEXT	As extracted; display strips leading zeros
<code>gross_amount</code>	REAL	
<code>total_fee</code>	REAL	
<code>net_amount</code>	REAL	<code>gross_amount - total_fee</code>
<code>source_pdf</code>	TEXT	Original filename
<code>ingested_at</code>	TEXT (ISO datetime)	
<code>match_status</code>	TEXT	<code>unmatched matched missing_from_invoice mismatch</code>
<code>invoice_line_id</code>	TEXT NULL	e.g. <code>AAE0319</code> when matched
<code>invoice_amount</code>	REAL NULL	Per-line amount from EFT

last_reconciled_at	TEXT NULL	
--------------------	-----------	--

Unique constraint: (batch_date, batch_number, net_amount) — dedupe key aligned with prototype recordKey logic.

7.3 Table: invoices

One row per ingested EFT PDF (file-level summary).

Column	Type	Notes
id	INTEGER PK	
invoice_number	TEXT	Summary line invoice #, e.g. 0600658
invoice_total	REAL	Summary amount
pdf_filename	TEXT	
processed_at	TEXT (ISO datetime)	
period_start	TEXT NULL	Min inv_date from lines
period_end	TEXT NULL	Max inv_date from lines

7.4 Table: invoice_lines

One row per AA-prefixed line on the EFT PDF.

Column	Type	Notes
id	INTEGER PK	
invoice_id	INTEGER FK → invoices.id	
invoice_line_id	TEXT	e.g. AAE0319
batch_number	TEXT	Extracted suffix
amount	REAL	Signed as on PDF; match uses abs(amount)
inv_date	TEXT (ISO date)	
match_status	TEXT	unmatched matched ambiguous
batch_id	INTEGER NULL FK → batches.id	Set on successful match

7.5 Table: reconciliation_runs (audit)

Column	Type	Notes
id	INTEGER PK	
invoice_id	INTEGER FK	
run_at	TEXT	
scoped_batch_count	INTEGER	Batches in date window
matched_count	INTEGER	
missing_from_invoice_count	INTEGER	
unmatched_line_count	INTEGER	
mismatch_count	INTEGER	
total_deposit	REAL	Sum matched gross_amount
total_fee	REAL	Sum matched total_fee
total_credit	REAL	Matched net total
credit_discrepancy	REAL	invoice_total - total_credit

7.6 Normalized internal record (application contract)

Unchanged from prototype — maps 1:1 to batches row before persistence:

```
site_id, batch_date, batch_number, gross_amount, total_fee, net_amount
```

Invoice extraction contract (from `eft_invoice.js`):

```
batchLines: [{ invoiceId, batchNumber, amount, invDate }]
summary: { invoiceNumber, amount }
```

7.7 Default filesystem layout (per machine)

```
~/Documents/Credit Batch Reconciler/
Stores/
  Sunset.db
  Mako.db
Exports/                ← default save location for Excel (user can change)
```

Configurable in app settings. **Do not** place live `.db` files on OneDrive for concurrent multi-writer use.

8. User interface

8.1 Primary screens

1. **Store selector** — list known stores (`.db` files in Stores folder); create new store; open store
2. **Dashboard** — batch count, last ingest date, recent reconciliations, outstanding flags
3. **Add Chevron PDF** — drag-and-drop; preview extracted rows; confirm append
4. **Add EFT PDF** — drag-and-drop; preview invoice lines; save + reconcile
5. **Reconciliation results** — matched table, exceptions panel, summary card
6. **Export** — save Excel report; optional “full ledger export”

8.2 Reconciliation results — flag types

Flag	Meaning	User action
Missing from invoice	Batch in scope has no EFT line	Contact processor; batch may not be paid
Unmatched invoice line	EFT line has no batch in DB	Upload missing settlement PDF
Amount mismatch	Matched pair but amounts differ	Investigate fees / duplicate batch #
Ambiguous match	Multiple batch candidates	Manual resolution (future) or fix data
Credit discrepancy	Invoice total $\neq$ sum of matched nets	Expected ~ 0 when complete; large gap = missing batches or coverage gap

8.3 Coverage indicator

Before treating Credit discrepancy as final, show:

“Invoice period {start}–{end}: **{N} batches in database, {M} lines on invoice, {X} missing from invoice, {Y} unmatched lines.**”

If batch corpus may be incomplete, display a warning.

9. Reconciliation logic

9.1 Scope

For a given `invoice_id`:

1. Compute `period_start` / `period_end` from `invoice_lines.inv_date` (min/max).
2. Select `batches` where `batch_date` is within `[period_start, period_end]` (± 0 day buffer; configurable later if needed).

9.2 Matching (invoice line → batch)

Reuse prototype rules from `reconcile.js` :

- Match on `stripLeadingZeros(batch_number)` and `round2(abs(line.amount)) === round2(batch.net_amount)`
- Disambiguate duplicate batch numbers by date proximity or exclude already-used batch rows (`recordKey` / `batch_id` consumption)
- One-to-one: each batch and each line used at most once per run

9.3 Reverse pass (batch → invoice line)

After forward matching, any **scoped batch** with `match_status` still `unmatched` → set `missing_from_invoice`.

9.4 Write-back

- Update matched `batches`: `match_status`, `invoice_line_id`, `invoice_amount`, `last_reconciled_at`
- Update `invoice_lines`: `match_status`, `batch_id`
- Insert `reconciliation_runs` summary row

9.5 Summary calculations

On **matched** batches only (same as prototype export summary):

Field	Formula
Total Deposit	Σ <code>gross_amount</code>
Total Fee	Σ <code>total_fee</code>
Total Credit	Total Deposit – Total Fee
Invoice Amount	<code>invoices.invoice_total</code>
Credit	Invoice Amount – Total Credit

10. Excel export specification

Export is generated from SQLite — **not** read back as ledger input.

10.1 Reconciliation report (primary export)

Sheet 1 — Matched batches (locked column layout from prototype):

Column	Source
Date	<code>batch_date</code> (grouped: blank on repeated dates)
Batch #	<code>batch_number</code>
Credit	<code>gross_amount</code>
Debit	<code>0.00</code>
Total	<code>gross_amount</code>
Fee	<code>total_fee</code>
Credit (net)	<code>net_amount</code>
T. Deposit	empty
T. Fee	empty
Invoice Amount	<code>invoice_amount</code>

Summary block below table: Total Deposit, Total Fee, Total Credit, Invoice Number, Invoice Amount, Credit.

Sheet 2 — Exceptions (new):

Column	Notes
--------	-------

Type	missing_from_invoice unmatched_line mismatch ambiguous
Batch #	
Batch Date	
Net Amount	
Invoice Line ID	
Invoice Amount	
Message	

10.2 Full ledger export (optional)

All `batches` rows with match status and invoice columns — flat table, no grouped dates, no summary formulas. For audit and OneDrive archival.

11. Build phases (1-6)

Each phase is shippable and testable on its own. Phases are cumulative.

Phase 1 — Desktop shell + SQLite foundation

Objective: Runnable Electron app with per-store database create/open and manual batch insert API (no PDF yet).

Deliverables:

#	Item
P1.1	Electron scaffold: <code>backend/main.js</code> , <code>backend/preload.js</code> , window loads <code>frontend/index.html</code>
P1.2	Stores folder resolution; create/open <code>{StoreName}.db</code>
P1.3	SQLite schema in <code>backend/reconciling/db/schema.sql</code> ; migrations in <code>backend/reconciling/db/migrations.js</code>
P1.4	Store module: <code>backend/reconciling/db/store.js</code> — <code>listStores</code> , <code>createStore</code> , <code>insertBatches</code> , <code>getBatches</code> , <code>getBatchCount</code>
P1.5	IPC handlers in <code>backend/main.js</code> ; <code>frontend/js/app.js</code> store selector + batch table view
P1.6	Dedupe on insert via unique constraint; return counts (added / skipped)
P1.7	Mac <code>.app</code> runs locally via <code>npm run start</code>

Success criteria:

- Create `Sunset.db` , insert test batches programmatically, reopen app, data persists
- Duplicate insert of same `(batch_date, batch_number, net_amount)` is skipped without error
- No PDF or Excel in this phase

Out of scope: Windows installer, PDF parsing

Phase 2 — Chevron PDF ingest

Objective: Upload Chevron settlement PDF → extract → append to active store database.

Deliverables:

#	Item
P2.1	Move prototype parsers to <code>backend/parsing/</code> (<code>ingestion.js</code> , <code>normalize.js</code> , <code>templates/chevron.js</code>)

P2.2	IPC: <code>parseChevronPdf(buffer)</code> in <code>main</code> → returns normalized batch records
P2.3	<code>frontend/</code> “Add Chevron PDF” dropzone + preview table
P2.4	On confirm: IPC <code>insertBatches</code> → <code>backend/reconciling/db/store.js</code>
P2.5	Status message: “Added 41 batches, 3 duplicates skipped”
P2.6	Parser warnings surfaced in UI
P2.7	Tests in <code>test/parsing/chevron-parser-regression.test.js</code>

Success criteria:

- Real Chevron PDF produces checksum-matching rows vs PDF printed totals
- Re-uploading same PDF does not duplicate rows
- `site_id` stored on each batch row

Reuses: Existing Chevron template validated in prototype

Phase 3 — EFT invoice ingest + storage

Objective: Upload EFT prenotification PDF → extract → persist `invoices` + `invoice_lines`.

Deliverables:

#	Item
P3.1	<code>backend/parsing/templates/eft_invoice.js</code>
P3.2	IPC: <code>parseEftPdf(buffer)</code> + <code>insertInvoice</code> via <code>backend/reconciling/db/store.js</code>
P3.3	<code>frontend/</code> “Add EFT PDF” dropzone + invoice preview
P3.4	Preview invoice header + line table before save
P3.5	Compute and store <code>period_start</code> / <code>period_end</code>
P3.6	Non-AA lines excluded from <code>invoice_lines</code> (summary only)
P3.7	Tests in <code>test/parsing/eft-invoice-parser.test.js</code>

Success criteria:

- EFT PDF saves invoice number, total, and all AA* lines
- Invoice visible in UI after save
- No reconciliation yet — lines stored with `match_status = unmatched`

Phase 4 — Bidirectional reconciliation + flags UI

Objective: On EFT save (or explicit “Reconcile” action), run full match, write statuses, display flags.

Deliverables:

#	Item
P4.1	<code>backend/reconciling/reconcile.js</code> — reverse pass for <code>missing_from_invoice</code>
P4.2	<code>backend/reconciling/reconcile-service.js</code> — orchestrates read → match → write-back
P4.3	IPC: <code>reconcileInvoice(invoiceId)</code> → matched, flags, summary
P4.4	<code>frontend/</code> reconciliation results screen: matched table + exceptions list
P4.5	Summary card: Total Deposit, Fee, Credit, Invoice Amount, Credit discrepancy
P4.6	Coverage indicator (scoped batch count vs invoice line count)

P4.7	Tests in <code>test/reconciling/reconcile.test.js</code>
------	--

Success criteria:

- Batch in DB within invoice period but not on EFT → flagged **missing from invoice**
- EFT line with no batch in DB → flagged **unmatched**
- Known sample pair from prototype reconciles with same totals as before
- Credit discrepancy matches prototype formula on matched rows

Breaking change from prototype: Export row set for reconcile report may include all scoped batches with status, not only matched rows (exceptions on separate sheet)

Phase 5 — Excel export

Objective: Export reconciliation report and optional full ledger to `.xlsx`.

Deliverables:

#	Item
P5.1	<code>backend/exporting/export.js</code> — reconciliation report + Exceptions sheet
P5.2	<code>backend/exporting/ledger-export.js</code> — optional full flat ledger export
P5.3	IPC: <code>exportReconciliation(invoiceId, savePath)</code> and <code>exportFullLedger(savePath)</code>
P5.4	Native save dialog from <code>frontend/</code> (default to Exports folder)
P5.5	Tests in <code>test/exporting/export.test.js</code> ; files open cleanly in Excel

Success criteria:

- Reconciliation export matches prototype column layout on Sheet 1
- Exceptions sheet lists all flags from last reconcile run
- User can save to OneDrive folder manually; file opens cleanly in Excel

Phase 6 — Legacy import + Windows distribution

Objective: Migrate existing store Excel workbooks; ship installable Windows `.exe`.

Deliverables:

#	Item
P6.1	<code>backend/exporting/legacy-import.js</code> — read legacy <code>.xlsx</code> (columns A-G) → records for <code>insertBatches</code>
P6.2	<code>frontend/</code> import UI: pick file, preview row count, skip duplicates, import report
P6.3	Best-effort handling of irregular legacy sheets (skip summary rows, blank date carry-forward)
P6.4	<code>electron-builder</code> config for Windows NSIS installer
P6.5	GitHub Actions workflow: Windows runner builds <code>.exe</code> artifact on tag
P6.6	<code>documentation/user-guide.md</code> — install, data folder, OneDrive guidance
P6.7	Optional: code signing (document in <code>documentation/</code> ; not blocking v1)

Success criteria:

- `Sunset Credit Batches for Nil.xlsx` (or equivalent) imports ≥95% of batch rows (A-G) without duplicates
- Windows installer runs on a clean Windows 10/11 machine without Node installed
- Imported batches participate correctly in Phase 4 reconciliation

Explicitly not imported: Scattered legacy invoice annotations in columns K-M (inconsistent); re-reconcile from EFT PDFs going forward

12. Phase summary

Phase	Name	User-visible outcome
1	Desktop shell + SQLite	App opens; pick store; batches persist in <code>.db</code>
2	Chevron PDF ingest	Drop settlement PDF → batches added to store
3	EFT invoice ingest	Drop invoice PDF → invoice stored
4	Reconciliation + flags	See missing batches, unmatched lines, Credit gap
5	Excel export	Clean <code>.xlsx</code> for review / OneDrive
6	Legacy import + Windows ship	Migrate old Excel; distribute <code>.exe</code>

13. Project structure (detailed)

13.1 Top-level layout

Folder	Purpose
<code>frontend/</code>	All renderer/UI code — HTML, CSS, client JS, assets. No SQLite, no parsers, no SheetJS.
<code>backend/</code>	Electron main process + three logic modules: parsing , reconciling , exporting .
<code>documentation/</code>	PRDs, architecture, extraction notes, user guides. Not loaded at runtime.

13.2 `frontend/` — renderer (UI only)

frontend/	
├ index.html	# App shell (evolved from batch_report_exporter.html)
├ css/	
│ └ app.css	# Styles (extracted from prototype <style> block)
├ js/	
│ └ app.js	# Boot, navigation, IPC orchestration
│ └ store-selector.js	# List / create stores
│ └ batch-ingest-ui.js	# Chevron upload + preview
│ └ invoice-ingest-ui.js	# EFT upload + preview
│ └ reconcile-ui.js	# Flags, summary card, coverage indicator
│ └ export-ui.js	# Save dialogs, export triggers
└ assets/	
└ icons/	# App icons (optional)

Frontend responsibilities:

- Render screens and tables
- Read PDF files via `<input type="file">`; send `ArrayBuffer` to main over IPC
- Display parse previews, reconciliation results, and warnings
- Trigger export/save via IPC; show native save path returned from main
- **Must not** import `better-sqlite3`, `xlsx`, or parser modules directly

13.3 `backend/` — main process + business logic

backend/	
├ main.js	# Electron entry: window, IPC registry, app paths
├ preload.js	# contextBridge – exposes typed <code>window.api.*</code> to frontend
├ parsing/	# PDF text extraction + processor templates
│ └ ingestion.js	# PDF bytes → lines (pdf.js)
│ └ normalize.js	# Raw template output → fixed batch schema

```

├── templates/
│   ├── chevron.js          # Chevron batch settlement PDF
│   └── eft_invoice.js       # EFT prenotification invoice PDF
├── reconciling/
│   ├── reconcile.js         # Pure match logic (batch ↔ invoice line)
│   ├── reconcile-service.js # Load from DB → reconcile → write statuses
│   └── db/
│       ├── schema.sql
│       ├── migrations.js
│       └── store.js         # CRUD: batches, invoices, lines, runs
├── exporting/
│   ├── export.js           # Generate .xlsx (write-only; never ledger input)
│   ├── ledger-export.js    # Reconciliation report + Exceptions sheet
│   ├── ledger-export.js    # Full flat batch dump
│   └── legacy-import.js    # One-time .xlsx → batch records (Phase 6)

```

Backend folder rules:

Folder	Owens	Must not
<code>parsing/</code>	PDF → normalized records / invoice result objects	Touch SQLite or Excel
<code>reconciling/</code>	SQLite <code>.db</code> per store, dedupe, bidirectional match, write-back	Know about PDF layout or Excel column headers
<code>exporting/</code>	Records / DB rows → <code>.xlsx</code> bytes or file on disk	Parse PDFs or run match logic

IPC surface (registered in `backend/main.js`, exposed via `backend/preload.js`):

Channel	Backend module
<code>stores:list</code> , <code>stores:create</code> , <code>stores:open</code>	<code>reconciling/db/store.js</code>
<code>batches:insert</code> , <code>batches:list</code>	<code>reconciling/db/store.js</code>
<code>invoices:insert</code> , <code>invoices:get</code>	<code>reconciling/db/store.js</code>
<code>parse:chevron</code> , <code>parse:eft</code>	<code>parsing/</code>
<code>reconcile:run</code>	<code>reconciling/reconcile-service.js</code>
<code>export:reconciliation</code> , <code>export:ledger</code>	<code>exporting/</code>
<code>import:legacy-xlsx</code>	<code>exporting/legacy-import.js</code>
<code>dialog:save</code>	<code>main.js</code> (Electron dialog)

13.4 documentation/

```

documentation/
├── PRD_credit_batch_reconciler.md # This document
├── PRD_batch_report_exporter.md   # Phase 1 prototype PRD (archived reference)
├── pdf_extraction_methods.md      # Parser implementation notes
├── architecture.md               # Layer diagram + IPC contract (Phase 1)
└── user-guide.md                 # Install, data folder, OneDrive (Phase 6)

```

13.5 test/ — mirrors backend layout

```

test/
├── parsing/
│   ├── chevron-parser-regression.test.js
│   └── eft-invoice-parser.test.js
├── reconciling/
│   ├── reconcile.test.js
│   └── store.test.js
├── exporting/
│   └── export.test.js
└── fixtures/
    ├── sample-chevron.pdf
    └── sample-eft.pdf

```

Root `package.json` scripts:

```

    {
      "main": "backend/main.js",
      "scripts": {
        "start": "electron .",
        "test": "node test/parsing/chevron-parser-regression.test.js && node test/parsing/eft-invoice-parser.test.js && node test/reconciling/reconcile.test.js",
        "build:mac": "electron-builder --mac",
        "build:win": "electron-builder --win"
      }
    }
  }
}

```

13.6 Migration from current prototype

Prototype path	New path
frontend/batch_report_exporter.html	frontend/index.html (UI split into frontend/js/*)
frontend/js/ingestion.js	backend/parsing/ingestion.js
frontend/js/normalize.js	backend/parsing/normalize.js
frontend/js/templates/chevron.js	backend/parsing/templates/chevron.js
frontend/js/templates/eft_invoice.js	backend/parsing/templates/eft_invoice.js
frontend/js/reconcile.js	backend/reconciling/reconcile.js
frontend/js/export.js	backend/exporting/export.js
frontend/test/*	test/parsing/* , test/reconciling/*
docs/pdf_extraction_methods.md	documentation/pdf_extraction_methods.md
PRD_batch_report_exporter.md	documentation/PRD_batch_report_exporter.md

Keep frontend/batch_report_exporter.html until Phase 2 UI is complete, then remove or move to documentation/ as archive.

13.7 Data files (runtime — not in repo)

```

~/Documents/Credit Batch Reconciler/
Stores/
  Sunset.db
  Mako.db
Exports/

```

Managed by backend/reconciling/db/store.js ; path configured in backend/main.js .

14. Risks and mitigations

Risk	Mitigation
Chevron PDF layout varies	Template isolation; regression tests on real samples
EFT invoice format varies	Validate against more stores before Phase 3 release
better-sqlite3 cross-compile for Windows	Build Windows .exe on Windows CI, not Mac cross-compile
Legacy Excel irregular layout	Phase 6 import rules + manual review report; don't import K-M invoice noise
User opens .db on OneDrive from two PCs	Document single-writer rule; Excel exports for sharing
Duplicate batch numbers same day	Match on batch_number + net_amount + batch_date ; flag ambiguous
Credit discrepancy trusted when corpus incomplete	Coverage indicator + warning in UI
Electron bundle size (~150 MB)	Acceptable for business desktop; document disk requirement

15. Open questions

Question	Impact	When to resolve
Does EFT PDF include <code>site_id</code> for auto store selection?	UX on invoice upload	Before Phase 3
Invoice amount matches <code>net_amount</code> (not gross)?	Match rule	Confirm with samples — prototype assumes net
Date scope buffer for reconciliation ($\pm$ days)?	False missing-batch flags	During Phase 4 testing
Manual resolution UI for ambiguous matches?	Phase 4 vs later	Defer to post-v1 unless samples require it
Auto-update mechanism for distributed <code>.exe</code> ?	Phase 6+	Optional; manual reinstall acceptable for v1

16. Success criteria (v1 complete — end of Phase 6)

- Operator installs Windows app without developer tooling
- Per-store `.db` accumulates all Chevron batches over time with deduplication
- EFT invoice ingest stores full line detail and runs bidirectional reconcile
- **Missing credit batches** are flagged when batches exist in DB but not on invoice
- Excel export provides clean matched table + exceptions sheet for client review
- Legacy Sunset-style workbook imports into `Sunset.db` for continuity
- All processing remains **local** — no data leaves the device

17. Relationship to existing prototype

The browser prototype (`frontend/batch_report_exporter.html`) validated:

- Chevron extraction (41 batches, checksum spot checks)
- EFT invoice parsing
- Invoice-to-batch matching and summary math

What changes in the desktop app:

Prototype behavior	Desktop app behavior
In-memory only	SQLite per store
Single batch PDF per session	Cumulative batch history
Invoice-driven row filter	Bidirectional flags
Excel is the only output	Excel is export snapshot
Browser file download	Native save dialog

What stays the same:

- Normalized batch schema
- Chevron and EFT templates
- Match key logic (`batch_number` + `net_amount` + date disambiguation)
- Excel column layout for reconciliation report

18. Testing strategy

Layer	Folder	Tests
PDF parsing	<code>test/parsing/</code>	Chevron + EFT parser regression
Reconciliation	<code>test/reconciling/</code>	Match logic + bidirectional cases

Storage	test/reconciling/	store.test.js — dedupe, migrations
Excel export	test/exporting/	Column layout, exceptions sheet
Integration	test/	PDF → parse → DB → reconcile → export (fixture PDFs)

Run `npm test` before each phase merge. Tests import from `backend/parsing/` , `backend/reconciling/` , and `backend/exporting/` — not from `frontend/` .

End of PRD